CHASE.AI: Agentic Contract-Based Hierarchical Design for High-Assurance Complex Engineered Systems

Pierluigi Nuzzo, Associate Professor, Department of Electrical Engineering and Computer Sciences, University of California, Berkeley

Requested funding: $80,000
Requested AWS Promotional Credits: $40,000

Abstract – Large language model (LLM) agents are increasingly employed to tackle the design of complex engineered systems, such as distributed cyber-physical systems, multi-robot systems, and integrated circuits. However, existing approaches lead to outcomes that are often unsatisfactory. Agents may propose to decompose a design into parts that are not fully compatible, tend to drift from the specifications over long horizon tasks, and leave no record that allows an error to be traced back to its root cause. In this project, we propose CHASE.AI, a framework for assured and traceable design of complex systems via decomposition based on assume-guarantee (A/G) contracts. In CHASE.AI, the agents propose a decomposition of a complex design task into contracts at each level of a design hierarchy, and a contract verification engine accepts a step only when it certifies that the lower-level contracts are implementable, and together refine the higher-level requirements. We will develop CHASE.AI by following a three-pronged approach. First, we will investigate representations of the design in terms of hierarchical contract networks that combine decomposition in space, over time, and across abstraction levels, and link every step of the design flow to verification and synthesis tools. Second, we will develop an agentic decomposition search method guided by a library of certified contracts. The method applies to multiple domains, and can be transferred to a new domain by instantiating the appropriate formalism, to express the contracts, and the corresponding algorithms and tools to reason about them. Finally, we will develop mechanisms enabling the agents to learn from the outcome of verification, where diagnoses such as counterexamples guide repair actions on the design specification or implementation, and verdicts serve as certified rewards to post-train the proposing models or design policies. CHASE.AI will extend CHASE, our open-source contract-based design framework, with LLM agents and will be evaluated in two application domains: integrated circuit design and warehouse multi-robot systems.

Keywords – Assume-guarantee contracts, compositional verification, distributed systems, GR(1), neurosymbolic reasoning, LLM agents, reinforcement learning with verifiable rewards, hardware formal verification.

Introduction

LLM agents are increasingly being employed to aid the design of complex engineered systems, from software and hardware systems to distributed multi-agent cyber-physical systems [1] such as multi-robot systems. Language models can help translate natural-language requirements into formal specifications and assertions expressed using temporal logics [2] or hardware description languages [3, 4]. However, existing approaches to AI-augmented design are prone to pitfalls and design errors, especially when dealing with the top-down decomposition of system-level specifications or bottom-up integration of component implementation, which are necessary steps in a design flow. Over long design tasks, agents may even disobey their task specifications or lose track of context information available from earlier design stages [5]. The reasoning behind a sequence of design decisions gets often buried in a set of transcripts, which makes it difficult to trace errors found late in the design process back to their root causes. As a result, AI-human collaboration in system design relies today on trial and error and expensive tests performed on the design artifacts at each design stage. A new generation of design methods would be essential to enable traceability and verifiability in AI-augmented design, and provide strong guarantees of correctness across components and abstraction levels.

Contract-based design can provide a formal foundation toward this goal. An A/G contract specifies what

a component guarantees under assumptions on its environment [6]. A decomposition is correct when the composition of the component contracts refines the system contract, that is, when any implementation of the composite contract can also implement the system contract in any valid environment. Existing contract-based tools such as CHASE [7] assume that contracts are written by human designers, while existing methods to automatically decompose specifications rely on a fixed library of components and contracts [8]. Decomposition approaches have also been recently pursued by relying on LLM reasoning capabilities: language models can search over a space of possible goal decompositions [9], compose tasks, validate them by tests [10], and delegate subgoals to various provers [11, 12, 13]. However, in existing LLM-based frameworks, verification steps are either informal, e.g., based on verbal reflection [14] and a limited number of tests, or they are limited to a single abstraction level in the design flow and a single formalism. Delegating to agents the task of writing contracts raises two challenges. First, specifications can be decomposed over different domains, e.g., in space, over time, and across levels of abstraction that use different formalisms, making it difficult to appropriately orchestrate different tools to ensure traceability. Second, the problem of decomposing temporal tasks into subtasks that are realizable is undecidable, in general, and often intractable, in practice, even if the system architecture, i.e., the system components and their interconnections, is given [15]. It is therefore desirable to devise mechanisms that can accelerate the search and prune the design space by leveraging all the certified invariants and the counterexamples found via verification methods throughout the design process.

The proposed framework, termed CHASE.AI, addresses these challenges via a three-pronged approach. The central principle of CHASE.AI is that agents propose and contracts certify: a step proposed by an agent is accepted only when a solver certifies it, so that guarantees established at higher abstraction levels propagate to every certified implementation. First, CHASE.AI develops representations of the design as hierarchical contract networks [16, 17] that combine decomposition in space, across abstraction levels, and over time, so that every step can be checked by existing tools (Thrust 1). Second, it develops an agentic decomposition search in which LLM agents, guided by a library of certified contracts, propose contracts at every level (Thrust 2). Finally, agents learn from verification outcomes by using diagnoses to repair failed decompositions and retaining certified contracts and reusable repair patterns (Thrust 3). CHASE.AI builds on our prior work on contract-based design methodologies [18] and on the CHASE(Contract-based Heterogeneous Analysis and System Exploration) framework [7], which connects a contract algebra to verification and synthesis tools and, recently, to LLM agents. Our work on contract-based architecture exploration [19, 20] has shown that infeasibility certificates can prune large design spaces, and their generalization via subgraph isomorphism [19] received the Best Paper Award at DATE 2024. Our timing contracts enabled consistency and availability analysis of distributed systems [1]. ACIES [21] and our ongoing work apply agent-proposed contracts to multi-robot systems and register-transfer level (RTL) verification. The same discipline applies to multi-agent LLM and vision-language-action systems, in which a parent agent decomposes a task and assigns contracts to sub-agents.

Proposed Research

The core of CHASE.AI is a neurosymbolic loop, in which LLM agents propose decompositions (Thrust 2), a contract verification engine certifies them against a hierarchical contract network (Thrust 1), and the outcome of verification returns to the agents to guide repair and learning (Thrust 3).

Thrust 1: Design Representations: Contract Networks. A complex design must be decomposed along the dimensions. Spatial decomposition splits a system into interacting components, as in our work on architectures [19], distributed systems [1], and multi-robot systems [21]. Hierarchical decomposition refines requirements across abstraction levels that use different formalisms, which our prior work relates through contract embeddings based on conservative approximations [16][17], so that a refinement proved on the embedded contracts also holds for the original ones. Temporal decomposition splits a mission into contracts

on consecutive time intervals, each enforced by a time-varying control barrier function [22]. Existing results address each dimension in isolation. Thrust 1 will develop representations of designs as contract networks, in which components are related by composition, by refinement across formalisms, and by sequencing in time, together with conditions under which the correctness of each local step implies the correctness of the whole network. The contracts themselves are domain-specific. Robot contracts couple a Generalized Reactivity(1) (GR(1)) mission layer [23] with controllers certified by control barrier functions [24] or reachability analysis, while learned policies, such as vision-language-action (VLA) models [25], are guarded at run time by monitors and safety filters. Hardware contracts relate the ports of RTL modules over clock cycles. Components may also be LLM-based agents, to which a parent agent assigns subtasks via contracts.

Thrust 2: Agentic Decomposition Search Methods. CHASE.AI will represent a design as a hierarchy of contracts that LLM agents build recursively, starting from the system requirements. At each level, an agent proposes contracts for the subcomponents, guided by certified decompositions of similar components retrieved from a library of certified contracts [26, 17] (Thrust 3), and the engine accepts the proposal only if the subcomponents together are certified to meet the parent contract. Each accepted subcomponent is either verified directly by a back end or handed to a new agent that works only on its own contract. When a component cannot meet its contract, the parent agent revises the decomposition and keeps the certified parts whose contracts are unchanged. Agents will explore alternatives by changing component boundaries, reassigning responsibilities among components, or introducing intermediate contracts at another abstraction level. The engine will prune candidates that violate compatibility or refinement conditions. Across domains, only the contract formalism and the back-end tools change.

Thrust 3: Evolving from Feedback of Verification. Every failed verification step returns a diagnosis that points to the responsible portions of the contracts, e.g., a counterexample trace or a minimal set of conflicting requirements. The agents access the tools through a Model Context Protocol (MCP) server. We will use these outcomes at three levels. First, diagnoses guide repair actions on the design specification or implementation, performed by formal methods [1] or by the agents. Second, recurring failures are generalized into patterns that exclude entire families of candidate decompositions [19] and are stored, together with the certified contracts, in libraries that evolve with each design [21]. Third, the verdicts of the engine serve as certified rewards to post-train the proposing models and the design policies via reinforcement learning. In all cases, the correctness of an accepted design never depends on the proposing agents.

Evaluation and preliminary results. Our prior work improves the reliability of auto-formalization by screening diverse candidate formalizations with solvers [27] and assesses reasoning agents with solver-backed checks [28]. For distributed systems, our timing contracts [1] verified and repaired the consistency and availability requirements of an automatic emergency braking system. In multi-robot systems, ACIES [21] decomposes mission requirements into GR(1) contracts, synthesizes controllers, and executes them under control barrier functions, and our ongoing work extends it with runtime monitors and a library of certified components. In hardware, component contracts will be written as SystemVerilog assertions (SVAs) and proved by hardware model checkers on open-source RTL designs (SVA expresses the same class of temporal properties as LTL in a hardware-oriented language). We will also consider a multi-agent LLM system as an additional case study. We will compare frontier models with and without verification-outcome feedback and retrieval against the post-trained proposer, including on faulty RTL designs, measuring the requirements certified beyond direct checking, the acceptance rate, solver runs and tokens per certificate, and library reuse.

Expected results

Months	Research result	Deliverable
1–3	Decomposition agent with verification outcomes, with contract embeddings across formalisms	Extended CHASE and MCP server, technical report
4–6	Formal, agent, and pattern-based repair with contract and pattern libraries	RTL benchmark with faulty variants, theory paper (e.g., CAV)
7–9	Decomposition search on RTL designs and warehouse multi-robot systems	Evaluation, open benchmarks and data, paper (e.g., FMCAD)
10–12	Proposer and VLA policies post-trained with certified rewards	Open-weight proposer, paper (e.g., NeurIPS), tutorial, talks to Amazon

Funds needed

Cash funding ($80,000). Partial support of one graduate student researcher (stipend and fees), and travel to present the results at conferences.

AWS Promotional Credits ($40,000). Amazon Bedrock for agent inference, Amazon SageMaker GPU instances for post-training an open-weight decomposition proposer and VLA policies (Thrust 3), Amazon EC2 instances for parallel runs of verification tools, and Amazon S3 for benchmarks and run traces.

Additional information

1. Application domain and analysis techniques. Compositional design of distributed systems, multi-robot systems, and integrated circuits, using A/G contracts, mixed-integer linear programming (Gurobi), linear temporal logic (LTL) model checking (nuXmv), satisfiability modulo theories (SMT) reasoning, GR(1) synthesis (slugs), and hardware model checking (SymbiYosys, Jasper). It covers sound neurosymbolic reasoning, LLM–solver interfaces, auto-formalization of contracts, and verification of distributed systems, robots, and hardware.

2. Current applications. CHASE has been used for requirement formalization and validation of cyber-physical systems [7], contract-based architecture exploration [19, 20], traceability validation of heterogeneous requirement hierarchies [16], consistency and availability analysis of distributed systems with Lockheed Martin [1], and multi-robot controller synthesis [21].

3. Potential applications to Amazon. (i) Distributed cloud services, where end-to-end latency, availability, and consistency budgets can be decomposed into per-component timing contracts [1], complementing models in TLA+ or P. (ii) Amazon Robotics fleets, where warehouse missions can be decomposed into contracts for small groups of robots. (iii) AWS processors and accelerators, where end-to-end properties of interconnects and coherence controllers can be decomposed into module-level SVA contracts.

4–5. Prior Amazon Research Awards. We did not receive any Amazon funding or AWS credits in the past three years.

6. Assumptions and soundness. A certified result is sound when the back-end tools return correct verdicts, the translations into the back ends preserve the semantics of the contracts, the mappings between formalisms are conservative, and the top-level specification, which agents cannot edit, is correct. Every contract is checked for consistency and compatibility, so that no refinement holds trivially.

7. Tools. *Code:* github.com/chase-cps/chase and github.com/chase-cps/chase-mcp. *License:* BSD 3-Clause. *On-boarding:* a website (chase-cps.github.io) with a usage guide, an API reference, and a Docker image. *Maintenance:* over 250 commits in the past three months, three active contributors.

Appendix A – References

[1] Xiao, Yifeng, et al. “Contract-Based Consistency and Availability Analysis for Distributed Cyber-Physical Systems.” MEMOCODE. 2026.

[2] Cosler, Matthias, et al. “nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models.” CAV. 2023.

[3] Orenes-Vera, Marcelo, Margaret Martonosi, and David Wentzlaff. “Using LLMs to Facilitate Formal Verification of RTL.” arXiv:2309.09437. 2023.

[4] Kang, Minwoo, et al. “FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware.” arXiv:2410.23299. 2024.

[5] Cemri, Mert, et al. “Why Do Multi-Agent LLM Systems Fail?” NeurIPS Datasets and Benchmarks Track. 2025.

[6] Benveniste, Albert, et al. “Contracts for System Design.” Foundations and Trends in Electronic Design Automation. 2018.

[7] Nuzzo, Pierluigi, et al. “CHASE: Contract-Based Requirement Engineering for Cyber-Physical System Design.” DATE. 2018.

[8] Iannopollo, Antonio, Stavros Tripakis, and Alberto Sangiovanni-Vincentelli. “Specification Decomposition for Synthesis from Libraries of LTL Assume/Guarantee Contracts.” DATE. 2018.

[9] Yao, Shunyu, et al. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” NeurIPS. 2023.

[10] Zelikman, Eric, et al. “Parsel: Algorithmic Reasoning with Language Models by Composing Decompositions.” NeurIPS. 2023.

[11] Jiang, Albert Q., et al. “Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers.” NeurIPS. 2022.

[12] Jiang, Albert Q., et al. “Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs.” ICLR. 2023.

[13] Ren, Z. Z., et al. “DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition.” arXiv:2504.21801. 2025.

[14] Shinn, Noah, et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” NeurIPS. 2023.

[15] Pnueli, Amir, and Roni Rosner. “Distributed Reactive Systems Are Hard to Synthesize.” FOCS. 1990.

[16] Naik, Nikhil Vijay, Alessandro Pinto, and Pierluigi Nuzzo. “Contract-Based Hierarchical Modeling and Traceability of Heterogeneous Requirements.” IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems. 2024.

[17] Wang, Timothy E., et al. “Hierarchical Contract-Based Synthesis for Assurance Cases.” NASA Formal Methods Symposium (NFM). 2022.

[18] Nuzzo, Pierluigi, et al. “A Platform-Based Design Methodology With Contracts and Related Tools for the Design of Cyber-Physical Systems.” Proceedings of the IEEE. 2015.

[19] Xiao, Yifeng, et al. “Efficient Exploration of Cyber-Physical System Architectures Using Contracts and Subgraph Isomorphism.” DATE. 2024.

[20] Xiao, Yifeng, and Pierluigi Nuzzo. “Contract-Based Architecture Exploration of Cyber-Physical Systems via Satisfiability Modulo Convex Programming.” DATE. 2026.

[21] Xiao, Yifeng, et al. “ACIES: A Self-Evolving Agentic Contract-Based Framework for Cyber-Physical System Control Synthesis.” REALM Workshop at EMNLP. 2026.

[22] Waqas, Muhammad, et al. “Contract-Based Control Synthesis with Barrier Functions for Vehicular Mission Planning.” CDC. 2022.

[23] Bloem, Roderick, et al. “Synthesis of Reactive(1) Designs.” Journal of Computer and System Sciences. 2012.

[24] Ames, Aaron D., et al. “Control Barrier Functions: Theory and Applications.” ECC. 2019.

[25] Kim, Moo Jin, et al. “OpenVLA: An Open-Source Vision-Language-Action Model.” CoRL. 2024.

[26] Iannopollo, Antonio, et al. “Library-Based Scalable Refinement Checking for Contract-Based Design.” DATE. 2014.

[27] Ni, Zhiyu, et al. “Draft-and-Prune: Improving the Reliability of Auto-formalization for Logical Reasoning.” arXiv:2603.17233. Code: github.com/zyni2001/draft-and-prune. 2026.

[28] Ni, Zhiyu, Yifeng Xiao, and Zheng Liang. “Agentified Assessment of Logical Reasoning Agents.” Agents in the Wild (AIWILD) Workshop at ICLR. 2026.